

# Anthony Ge

+1 (914) 770-5675 | [anthonykounge@gmail.com](mailto:anthonykounge@gmail.com) | [github.com/SomeGuy966](https://github.com/SomeGuy966) | [linkedin.com/in/anthony-ge/](https://linkedin.com/in/anthony-ge/)

## Education

**Northwestern University** | Evanston, IL

**Expected June 2028**

*Bachelor of Science, Computer Science and Mathematics (Double Major)*

GPA: 3.95/4.00

- Courses: Probability & Stochastic Proc (3-quarter sequence), Real Analysis, Optimization, ML, Applied Statistical Learning

## Experience

### Amazon

**Jun. 2026 – Sep. 2026**

*Software Development Engineer Intern, Prime Video*

*Seattle, WA*

- Cut defect-discovery latency on Prime Video's catalog export feeds from days or weeks to ~5 minutes as sole author of an event-driven data-integrity monitoring service (Java 17, AWS Lambda, SNS→SQS; 12 CloudWatch metrics, 14 auto-ticketing alarms)
- Surfaced 2,465 XSD schema violations in the first 3 hours of production traffic and 17,152 missing-required-field violations traced to a producer-side gap; verified against production bytes that 98.3% of 4,848 violations sat in a single field
- Delivered ~16,600 lines across two services (56% tests; 398 tests behind a 98% line/branch coverage gate) in 51 solo code reviews over ~7 weeks; all infrastructure in TypeScript CDK across 4 accounts and 2 regions on one self-mutating pipeline

### MedSphere

**Jan. 2026 – Mar. 2026**

*Software Engineer Intern*

*Skokie, IL*

- Built document-to-graph pipeline extracting pharma/biotech text/metadata into graph nodes and edges
- Implemented entity extraction and normalization (canonicalization, deduping, synonym resolution) for consistent schema at scale
- Built relation extraction and graph construction for confidence-weighted typed edges and source-backed explanations
- Productionized graph builds via async jobs for batch/incremental updates, plus logging/monitoring and stable APIs for consumers

## Projects

### VIX Probabilistic Forecasting – Walk-Forward Evaluation

[github.com/SomeGuy966/implicit-volatility-forecasting](https://github.com/SomeGuy966/implicit-volatility-forecasting)

- Built a Python package (arch) forecasting the CBOE VIX 1 and 5 days ahead as full predictive distributions: random walk, EWMA, HAR, AR(1)-GARCH(1,1)-t and HAR-GARCH(1,1)-t, each refit at 676 no-look-ahead walk-forward origins (2024–26)
- Evaluated with Diebold–Mariano (HAC/HLN), Kupiec and Christoffersen coverage tests, Winkler score, and pinball loss
- Found RMSE gains over the random walk of only 2.5% at  $h=1$  (HAR-GARCH-t; significant under absolute loss only) and 8.5% at  $h=5$  (HAR, DM  $p<0.05$ ); only the GARCH-t intervals pass Christoffersen independence, scoring 6–10% better
- Shipped as a typed package: CLI, joblib-parallel backtest (3,380 fits/horizon in ~20 s), 62 pytest tests incl. a no-look-ahead test per model and a Monte Carlo size check of the DM test, mypy --strict, CI, byte-for-byte reproducible results

### Kalshi Weather-Market Forecasting | Python, LightGBM, pytest, mypy-strict, CI

[github.com/SomeGuy966/weather-bot](https://github.com/SomeGuy966/weather-bot)

- Built a leakage-safe pipeline turning hourly METARs and NOAA NBM/GFS guidance into calibrated daily-high distributions for 8 U.S. cities with Kalshi markets: 586K hourly snapshots (2020–26) via strict as-of joins and chronological train/calibrate/test splits
- Trained a calibrated 31-class LightGBM model of the high vs. a forecast/running-max anchor, cutting log-loss 30% against NBM's own probabilistic guidance (1.49 vs. 2.12; ECE 0.004); mapped it onto Kalshi contract ladders for real-time partial-day inference
- Archived 3.2K contracts (order-book candles, trade tape) and backtested 52K contract-hours after fees; the market was sharper at every hour and city (Brier 0.120 vs. 0.141); shipped as a documented negative result: paper-trading ledger, no capital deployed

### S&P 500 ARIMA vs. Random Walk – Walk-Forward Forecast Evaluation

[github.com/SomeGuy966/sp500\\_arima](https://github.com/SomeGuy966/sp500_arima)

- Built a walk-forward evaluation of ARIMA forecasts on 50 years of daily S&P 500 data (600 monthly re-fits, 12.6k out-of-sample days), benchmarked against random-walk and random-walk-with-drift baselines using Diebold–Mariano tests
- Showed the AIC-selected ARIMA's 2.48% monthly MAPE is statistically indistinguishable from a random walk (DM  $p = 0.93$ ) and significantly worse at a one-day horizon ( $p = 0.001$ ); 95% prediction intervals cover only 91% of one-day outcomes
- Backtested a daily sign-of-forecast strategy with transaction costs (gross Sharpe 0.70 vs. 0.60 buy-and-hold, falling to 0.21 at 5 bps) and traced the gross edge to the index's lag-1 autocorrelation flipping from +0.08 (1975–89) to –0.12 (2005–24)
- Engineered as a typed Python package (statsmodels, pandas): CLI, parallel walk-forward engine, 59 tests at 95% coverage, mypy --strict, GitHub Actions CI on Python 3.11–3.13

### BTC-USD Order-Flow Research Pipeline (C++/Python)

[github.com/SomeGuy966/btc-trading-pipeline](https://github.com/SomeGuy966/btc-trading-pipeline)

- Built a hybrid C++/Python research pipeline for Gemini BTC-USD: REST ingest (Python and C++ implementations behind one pybind11 interface), header-only C++ order-flow features, and online Lasso inference refit every tick on a rolling 10-tick window
- Diagnosed and fixed a state-leakage bug where a rolling-window feature was re-fed on every model refit; added regression tests
- Profiled the Python/C++ boundary: per-feature pybind11 calls ran at  $0.66\times$  pure Python; batching all four features into a single crossing and passing trades by const reference reached  $2.32\times$  Python ( $3.5\times$  over per-call), equivalence-verified before timing
- Built JSONL record/replay for deterministic offline runs and a one-command CMake/Conan/Ninja/Poetry build; four GitHub Actions gate every PR on pytest, ruff/mypy strict, GoogleTest, and clang-format/clang-tidy (85 tests)

## Skills

**Languages:** Python, C/C++, Java, MATLAB, SQL, TypeScript/JavaScript, Lua

**Quant & ML:** statsmodels, arch (GARCH), LightGBM, CatBoost, PyTorch, TensorFlow, pandas, NumPy, SciPy, joblib

**Dev Tools:** Git, GitHub Actions, AWS (Lambda, S3, CloudWatch, CDK), pybind11, CMake, Conan, pytest, GoogleTest, mypy